Reprint of paper to be cited as:
Radu Dogaru,Ioana Dogaru,and Robert-CristianTecaru, "A Comparison of Several MCU-Oriented TinyML Models for Skin Lesions Classification", in Advances in Digital Health and Medical Bioengineering II, Springer Nature Switzerland AG, H.Costin et al.(Eds.): EHB2025,144, pp.1–8, 2026.

A Comparison of Several MCU-Oriented TinyML Models for Skin Lesions Classification

Radu DOGARU[1§], Ioana DOGARU[1], and Robert-Cristian TECARU[1]

[1]National University of Science and Technology POLITEHNICA, Bucharest, Romania
radu.dogaru@upb.ro, ioana.dogaru@upb.ro, robert.tecaru@stud.etti.upb.ro

Abstract. This paper investigates three TinyML models (including a novel one, entitled HVTINY), and compares them against state-of-the-art solutions for the problem of skin cancer detection and diagnosis. Results presented herein indicate that such compact models fit well into low cost, portable MCU-based platforms like ESP32-CAM, thus ensuring a cheap and convenient solution to build portable medical devices for early screening in primary care or remote areas. The results for some widely know datasets (a binary one, for detecting malign lesions and a dataset for diagnosis classification) indicates state of the art performance (93% accuracy for malign lesion detection, and more than 98% for skin-lesions classification) comparable to what is obtainable with more complex models, associated with more costly medium size computational platforms.

Keywords: Skin-lesion classification, Tiny-ML, Micro-Controller-Unit, Convolutional Neural Networks, Resources-constrained devices.

1 Introduction

The problem of skin cancer detection and classification is a well known and widely recognized one. As indicated in [1] early detection is critical and could save lives if performed with portable and cheap instruments to scan lesions and predict accurately suspect clinical conditions (e.g., malignant formations). Consequently, in recent years various machine learning algorithms and applications were developed [2][4][7], along with competitions and datasets (e.g., [3],[5]) containing a wide category of professionally labeled images.

While mobile platforms (mobile phones, tablets) can be regarded as of medium complexity, being capable to accommodate machine learning models with sizes in the range of tens-hundreds Mbytes, a recent challenge [7] is to include intelligent recognizes into low-cost low-power MCU-based TinyML platforms (with less than 512 Kbyte RAM and several Mbyte FLASH and operating frequencies in the range of hundreds of MHZ) such as ESP32-CAM [10]. Such devices (also equipped with WiFi

connectivity, color camera, and flashlight) are perfectly suited to bring early screening closer to primary care or remote areas, using non-invasive techniques.

In [4] skin-cancer classification on a widely known multi-class dataset [5] is approached using a recent architecture called VRES-CNN [6] specially developed to fit resources constraint platforms. Although results are quite good (up to 98.55% accuracy), competing favorably with more complex CNN models [2], such models are still large for MCU-based solutions, but remain best option for Android or other operating systems supporting mobile phones or tablets.

In this work we address, with positive answer, the question of whether accurate skin cancer detection and classification may fit MCU-based low-cost platforms. To our knowledge, there is so far no solution reported for skin-lesion detection using low-power and low-cost platforms such as ESP32-CAM or similar ones. Only recently, image classification tasks on MCU platforms were reported for gray-level fashion articles images recognition [8] or pneumonia detection [9], the latter being closer to medical applications. One major problem for TinyML models is that they should be accurate enough yet have exceedingly small sizes to fit the reduced resources for FLASH (to store the trained model) and RAM (to store the variables associated with neurons activations in various layers) available on these devices. In addition, they should have a limited number of computations (MAC = multiply and accumulates) to achieve reasonable latencies (response times to input images), with less than one second.

Section 2 of this work introduces the TinyML models considered in this study and describe them in more detail, from the perspective of their effective deployment using the tools from LiteRT[1] . Herein, to achieve maximal compression, we considered INT8 post training quantization, and all reported accuracies are for the quantized models. The two datasets explored in this study (one for binary classification and the other for multi-class diagnosis) are detailed. Section 3 provides a synthesis of the experimental results emphasizing pros and cons for each model and providing a comparison basis, including the more complex VRES-CNN model [6]. The concluding remarks complete this section.

2. Tiny ML models and datasets

In recent years various TinyML CNN models were proposed. Herein we consider at least three candidates and evaluate them for two relevant datasets in skin cancer diagnosis: *i) the first one*, called next ***mel_skin_48*** corresponds to the *binary problem of detecting malignant versus benign* skin lesions. The original set[2] (available as a collection of train and test folders for both classes) was converted into tensor format with image shapes of 48x48x3. The set is balanced and consists of 10000 image samples divided into 9632 train and 1024 validation samples (Fig.1); For this dataset (with images extracted from a previous ISIC competition) there are 79 public codes available in Kaggle, among them 93% accuracy is reported using a MobileNetV2 (pretrained) medium complexity model.

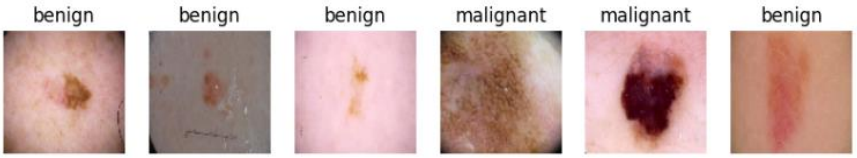

Fig. 1. Samples from the mel_skin_48 dataset

A similar set is considered in [2] where the best validation accuracy reported is 91% using a rather complex model (Xception+StackingCV classifier), not suitable for MCU; *ii) the second dataset* ***skin_cancer_balanced***, is a balanced version (achieved using over-sampling) of the HAM1000 dataset [5]. Here, the image size is 28x28x3 and there

[1] https://github.com/google-ai-edge/LiteRT

[2] https://www.kaggle.com/datasets/hasnainjaved/melanoma-skin-cancer-dataset-of-10000-images

are 7 classes associated with various skin lesions. Fig.2 displays the distribution of samples per class in the original validation set, while the resulting balanced validation set has around 1700 samples per class (a total of 11734 validation samples and 35201 samples for training). The same balanced set was evaluated in other works such as [4] or in [7] where 98.25% validation accuracy was reported for a medium-complexity computing platform (Raspberry-Pi 5).

original (not-balanced) validation set

samples	class	medical description of the class
(327,)	0:	('akiec', 'Actinic keratoses and intraepithelial carcinomae'),
(514,)	1:	('bcc', ' basal cell carcinoma'),
(1099,)	2:	('bkl', 'benign keratosis-like lesions'),
(115,)	3:	('df', 'dermatofibroma')}
(6705,)	4:	('nv', ' melanocytic nevi'),
(142,)	5:	('vasc', ' pyogenic granulomas and hemorrhage')
(1113,)	6:	('mel', 'melanoma'),

Fig. 2. Distribution of samples in the non-balanced **skin_cancer** dataset

The Tiny-ML models considered herein share a common characteristic: they have a specific number of convolutional *macro-blocks* (each including certain 2D-convolution operators) *followed by a simple classifier* (usually a single Dense layer). There are, however, certain differences as explained next. To get a better compromise between accuracy and device restrictions, and inspired by VRES-CNN [6], a simplified, compact model entitled HVTINY is first time proposed and evaluated in this work[3]. The other two models, already described elsewhere, were adapted into a unique programming and deploying framework designed as a Jupyter notebook running in Kaggle.

2.1 The LB-CNN model

Herein we used an improved, novel implementation of the idea exposed in [13]. As seen in Fig.3 the specific model presented there has 3 convolutional macro-blocks (in general they can range between 1 and 3), each being composed of a *depth-wise convolutional layer* with a certain factor of multiplication for the input channels. For the particular example considered in Fig.3, the relevant set of hyperparameters is [8,4,2] thus indicating that in the first macro-layer there are 3*8=24 binary convolutional kernels (filters), then 24*4=96 and finally 192 filters in the end. Specific to LB-CNN is the *binary nature of the convolution* making it suitable for dramatic reduction in complexity and energy when using specialized libraries as proposed in [12] for MCU or FPGA deployment. Also particular to LB-CNN is the fast design of the binary convolution kernels in a loop with enough number of trials (usually 100 trials, each lasting no more than a second) where binary {-1,1} values are assigned randomly (depth-wise layers are not trained at all) and the output classifier is rapidly trained with the ELM (extreme learning machine) algorithm. The best accuracy solution among all trials is identified and the corresponding binary kernels are saved. To improve accuracy, the classifier layer can be additionally trained using Keras optimizers (e.g. Adam). Since only the output Dense layer is trainable, the entire training process is fast and convergent. To conclude, the LB-CNN offers two advantages; namely, a very convenient chain of convolutions (binary weights remove the need to use multipliers and require low memory storage) and exhibit fast training even when no GPU is available. However, tuning the model filters is necessary to optimize overall performance.

The EdgeImpulse[4] framework is used to evaluate the necessary RAM and FLASH resources for a given MCU-based platform. For the target platform considered here (ESP32-CAM/Eye) one may consider 256 Kbyte RAM and 4Mbyte FLASH as limiting factors and from Figure 3 it is *quite clear that this particular LB-CNN model fits well*. The latency, however, is large (1.7 seconds) due to the substantial number of inputs in

[3] Code for the HVTINY model is available: https://github.com/radu-dogaru/hvtiny
[4] https://studio.edgeimpulse.com/

the output classifier (6912) and it can be reduced by a proper choice of the filter multiplier set (as seen in Table 1).

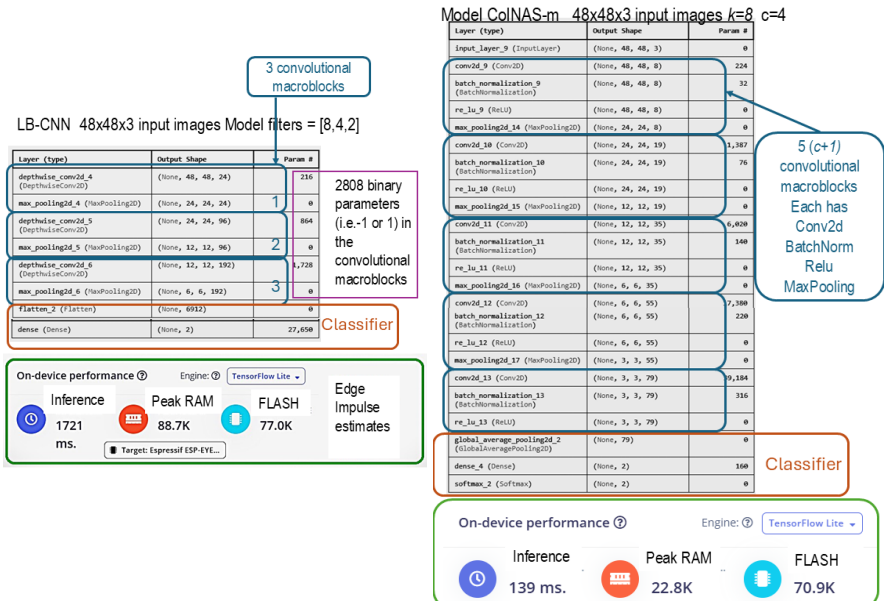

Fig. 3. The LB-CNN (left) and Col-NAS (right) models and their implementation in an ESP-Eye device. The most accurate *tflite* files for these models are available via github[3]

2.2 The Col-Nas model

In recent work [11], an extremely useful framework for designing MCU-oriented CNNs is provided, with a focus on using Neural Architecture Search (NAS) to optimize accuracy while fitting the RAM and FLASH constraints of the MCU units. In [11] MCUs from the STM company[5] were considered but their proposed model (described here as **Col-NAS**) may be expanded to other MCUs as well. As seen in Figure 3, the model also involves a number of *convolutional macroblocks* (fixed by the *c* hyperparameter) with a number of filters *k* in the first level and a particular algorithm to multiply the number of filters for each additional level (macro-block) with a certain precomputed value. The output classifier is also a simple one, using a Dense layer. The pair *(c,k)* is tuned to optimize the performance while keeping the model under memory restrictions associated with the given MCU. Although the model has a larger number of parameters its implementation is rather convenient, and in particular it is useful to note a very good (small) latency of around 0.3 seconds and the low RAM occupancy (RAM is usually the most problematic resource in a MCU unit).

2.3 The HVTINY model

Given the experience with VRES-CNN model [6] providing good accuracy at the expense of large latencies (many multiply-and-accumulate = MAC) we performed a careful analysis of the limiting factors and decided to propose a MCU-oriented model while keeping the main idea of versatility (possibility to choose among many choices for filter hyper-parameters). The description of the resulting HVTINY model is depicted in Fig.4. Unlike in VRES-CNN, residual connections were eliminated (reducing 2 times the MACs), the *nl* list (nonlinearity degree of macro-blocks) is also eliminated (only the first nonlinear macro-block) exhibits *nl=2* nonlinearity [6]) and normal convolutions were replaced with separable and depth-wise convolutions in the remaining *c* macroblocks. Consequently, the HVTINY models are defined by *k* and *c* with a similar meaning as in the case of Col-NAS model and a multiplier "profile" (e.g., [1,4,2,1]) with *c+1* elements indicating the multipliers of *k* and consequently the

[5] https://www.st.com/content/st_com/en.html

number of convolutional kernels (filters) per each macro-block.

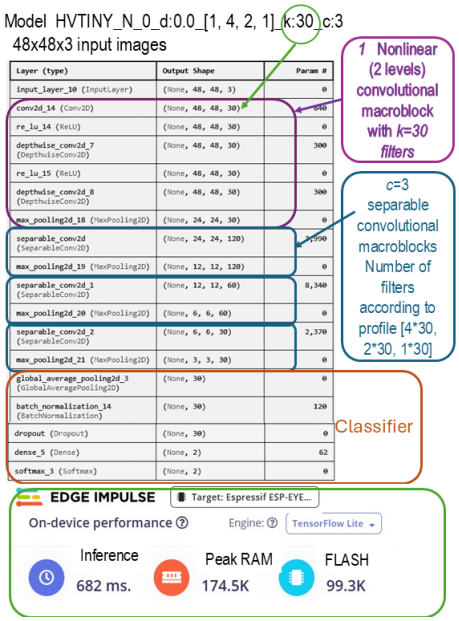

Fig. 4. The HVTINY model and its implementation in an ESP-Eye device.

3. Experimental results and conclusions

All previously introduced TinyML models were optimized (hyper-parameter tuning) and trained with respect to the two datasets introduced in section 2. Table 1 indicates the best selection for each model including the device occupancy and latency as evaluated via EdgeIpulse for the ESP-Cam/Eye target. For reference, the best result on the same dataset using VRES-CNN is presented in the last row.

Table 1. Best results for the TinyML models, in the case of mel_skin_48 dataset

Model type	Relevant hyperpars	.tflite size (kbyte)	INT8 Validation accuracy (%)	RAM (Kbyte)	FLASH (Kbyte)	Latency (ms)
LB-CNN	[2,2,2]	7	**92.77**	29.35	55.4	458
Col-NAS	(k,c)=(3,4)	18	92.38	22.8	70.9	195
HVTINY	(k,c)=(30,3) [1,4,2,1]	38	92.57	174	99.3	681
VRES	d=0.6 [40,120,60,40] [2,1,1,0]	94	95.11	348	161	1903

Although best performance (95.11% validation accuracy) is related with VRES-CNN, *the RAM occupancy exceeds the capability* of the MCU-based platform. For such platforms, accepting a sacrifice in accuracy LB-CNN would perform quite well with almost 93% validation accuracy. For a better latency (only 0.2 seconds) one may consider the Col-NAS model, with a lower accuracy. For the case of **skin_cancer_balanced** dataset with 7 classes, the best models are summarized in Table 2. Note that in this case the LB-CNN model performs badly while the best compromise is achieved by the HVTINY model. Given the reduced size of the images (28x28) now the VRES-CNN model fits the MCU-constraints but gives overall lower performance.

To conclude, skin-lesion detection and classification is possible on low-cost MCU-based platforms given that a proper TinyML CNN model is selected and low image resolutions are used (at most 48x48). HVTINY model ensures a compromise between accuracy and resources in most cases, but in general it is also good to consider the other two or more TinyML models to expand the selection base. Further research will consider other relevant datasets.

Table 2. Best results for the TinyML models, in the case of **skin_cancer_balanced** dataset

Model type	Relevant hyperpars	.tflite size (kbyte)	INT8 Validation accuracy (%)	RAM (Kbyte)	FLASH (Kbyte)	Latency (ms)
LB-CNN	[4,3,2]	16	58.01	19.6	64.3	317
LB-CNN	[16,2]	41	86.61	60.4	80.7	420
Col-NAS	(k,c)=(20,3)	154	97.85	30.8	203	256
HVTINY	**(k,c)=(30,3) [1, 4, 4, 1]**	**51**	**98.14**	**68.8**	**111.7**	**321**
VRES	d=0.6 [40,120,60,40] [2,1,1,0]	94	97.57	135	161	1013

References

1. Esteva A, Kuprel B, Novoa RA, Ko J, Swetter SM, Blau HM, et al. Dermatologist-level classification of skin cancer with deep neural networks. Nature. 2017;542(7639):115–8. https://doi.org/10.1038/nature21056.
2. Bassel A, Abdulkareem AB, Alyasseri ZAA, Sani NS, Mohammed HJ. Automatic malignant and benign skin cancer classification using a hybrid deep learning approach. Diagnostics. 2022; 12:2472
3. Kassem, M.A.; Hosny, K.M.; Fouad, M.M. Skin lesions classification into eight classes for ISIC 2019 using deep convolutional neural network and transfer learning. IEEE Access 2020, 8, 114822–114832.
4. Armanda DM., Cococi AG.,Dogaru I., Dogaru R. (2024), "Skin Cancer Detection Using Constrained Neural Networks and the Development of an Optimized Technical Infrastructure for Resource-Efficient Diagnosis" 2024 E-Health and Bioengineering Conference (EHB).
5. Tschandl P., Rosendahl C., & Kittler H., (2018). The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. Scientific Data, 5(180161). https://doi.org/10.1038/sdata.2018.161 and https://www.kaggle.com/datasets/utkarshps/skin-cancer-mnist10000-ham-augmented
6. Dogaru R. and Dogaru I (2024), "VRES-CNN: A Tiny Convolutional Image Classifier with Versatile Choice of Hyperparameters," 2024 16th International Conference on Electronics, Computers and Artificial Intelligence (ECAI), Iasi, Romania, 2024, pp. 1-6, doi: 10.1109/ECAI61503.2024.10607405.
7. Vincent, G. Darian, and N. Surantha, "Performance Evaluation of Convolutional Neural Network (CNN) for Skin Cancer Detection on Edge Computing Devices," Applied Sciences, vol. 15, no. 6, Art. no. 6, Jan. 2025, doi:10.3390/app15063077.
8. Ben Aicha, F. Optimized embedded AI: efficient implementation of CNNs on ESP32-CAM for real-time image classification. *Computing* **107**, 206 (2025). https://doi.org/10.1007/s00607-025-01559-z
9. Abd Djawad, Y., Rifqie, D. M., Ridwansyah, R., Supriadi, J., Sokku, S. R., Kaswar, A. B., & Idris, M. (2025). Performance Analysis of Convolutional Neural Network for Pneumonia Detection on an Embedded AI System. *Ingeniería y universidad*, *29*, 3.
10. Dietz, H., Abney, D., et.al (2022). ESP32-CAM as a programmable camera research platform. Imaging, 232, 2.
11. Garavagno A.M., Leonardis D., Frisoli A. (2024), ColabNAS: Obtaining lightweight task-specific convolutional neural networks following Occam's razor, Future Generation Computer Systems, Volume 152, 2024, p.152-159, https://doi.org/10.1016/j.future.2023.11.003.
12. Sakr, F., Berta, R., Doyle, J., Capello, A., Dabbous, A., Lazzaroni, L., Bellotti, F.: CBin-NN: An Inference Engine for Binarized Neural Networks. Electronics. 13, 1624 (2024). https://doi.org/10.3390/electronics13091624.
13. Dogaru, R., & Dogaru, I, "LB-CNN: An Open Source Framework for Fast Training of Light Binary Convolutional Neural Networks using Chainer and Cupy", 2021, https://arxiv.org/abs/2106.15350